# EDA & Data Wrangling — ATP Tennis Bayesian Model

Jensen Suhenda 44377570

```r
library(dplyr)
library(tidyr)
library(lubridate)
library(ggplot2)
library(knitr)
```

## Data Loading

```r
url <- paste0(
  "https://raw.githubusercontent.com/",
  "jsuhenda/bayesian-tennis-model/",
  "refs/heads/main/atp_tennis.csv"
)
df <- read.csv(url)
cat("Rows:", nrow(df), "| Cols:", ncol(df), "\n")
```

```
## Rows: 67196 | Cols: 17
```

## Raw Data Glimpse

```r
glimpse(df)
```

```
## Rows: 67,196
## Columns: 17
## $ Tournament <chr> "Australian Hardcourt Championships", "Australian Hardcourt~
## $ Date       <chr> "2000-01-03", "2000-01-03", "2000-01-03", "2000-01-03", "20~
## $ Series     <chr> "International", "International", "International", "Interna~
## $ Court      <chr> "Outdoor", "Outdoor", "Outdoor", "Outdoor", "Outdoor", "Out~
## $ Surface    <chr> "Hard", "Hard", "Hard", "Hard", "Hard", "Hard", "Hard", "Ha~
## $ Round      <chr> "1st Round", "1st Round", "1st Round", "1st Round", "1st Ro~
## $ Best.of    <int> 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3,~
## $ Player_1   <chr> "Dosedel S.", "Clement A.", "Escude N.", "Knippschild J.", ~
## $ Player_2   <chr> "Ljubicic I.", "Enqvist T.", "Baccanello P.", "Federer R.",~
## $ Winner     <chr> "Dosedel S.", "Enqvist T.", "Escude N.", "Federer R.", "Fro~
## $ Rank_1     <int> 63, 56, 40, 87, 81, 105, 26, 218, 24, 351, 55, 155, 208, 79~
## $ Rank_2     <int> 77, 5, 655, 65, 198, 58, 51, 11, 129, 134, 21, 15, 122, 142~
## $ Pts_1      <int> -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1,~
## $ Pts_2      <int> -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1,~
## $ Odd_1      <dbl> -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1,~
## $ Odd_2      <chr> "-1", "-1", "-1", "-1", "-1", "-1", "-1", "-1", "-1", "-1",~
## $ Score      <chr> "6-4 6-2", "3-6 3-6", "6-7 7-5 6-3", "1-6 4-6", "7-6 5-7 6-~
```

# Wrangling

We reduce to the four variables needed by the Bradley-Terry model: winner, loser, surface, and year.

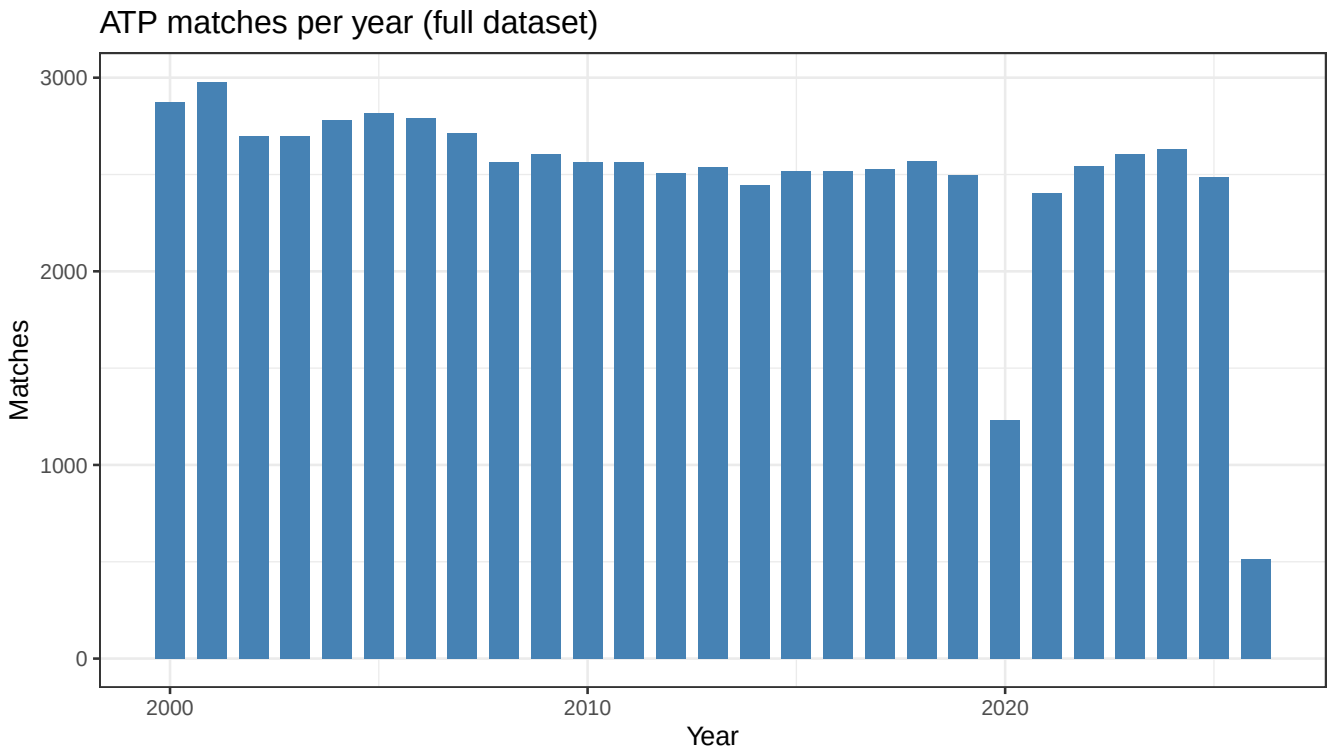
```
df_clean <- df %>%
  mutate(
    winner_name = Winner,
    loser_name  = ifelse(Winner == Player_1, Player_2, Player_1),
    surface     = Surface,
    date        = as.Date(Date),
    year        = year(date)
  ) %>%
  select(winner_name, loser_name, surface, year) %>%
  filter(
    !is.na(winner_name), !is.na(loser_name),
    !is.na(surface),     !is.na(date)
  )
cat("Clean rows:", nrow(df_clean), "\n")
```

```
## Clean rows: 67196
```

# EDA: Full Dataset

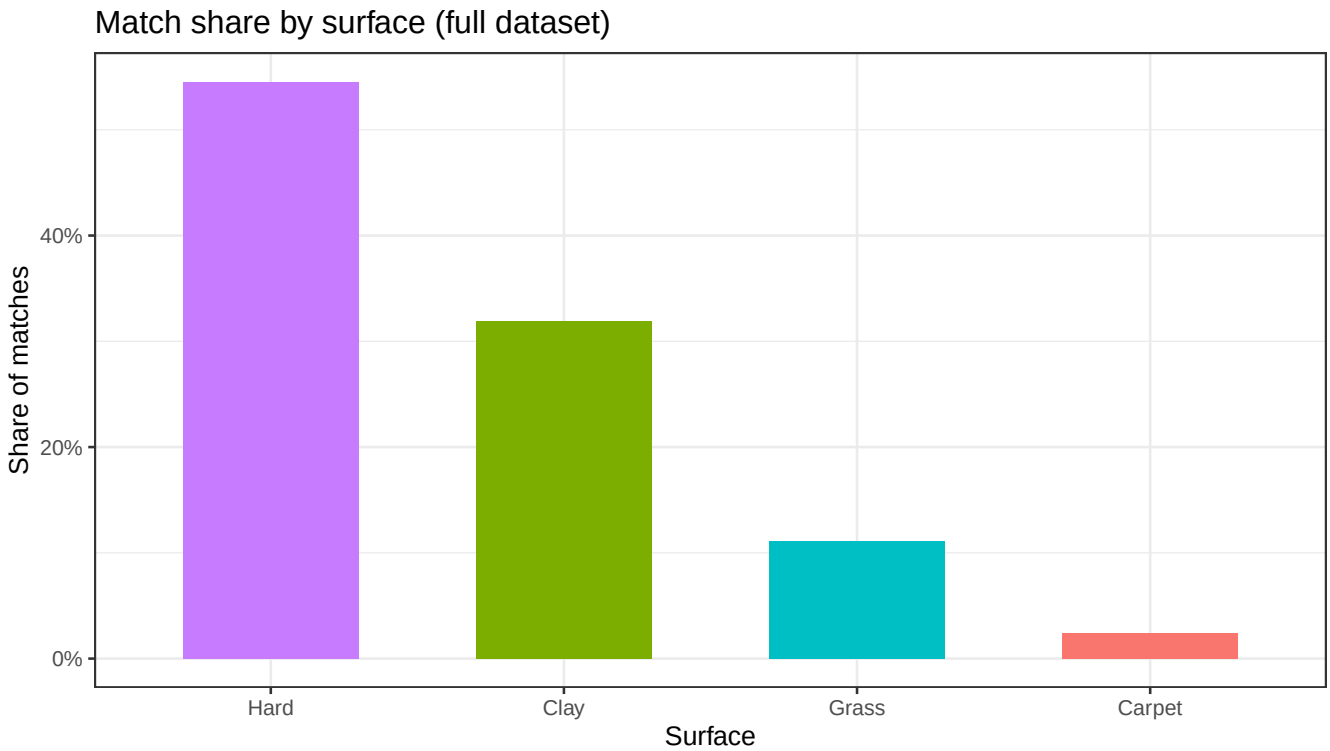
## Matches per year

```
df_clean %>%
  count(year) %>%
  ggplot(aes(x = year, y = n)) +
  geom_col(fill = "steelblue", width = 0.7) +
  labs(x = "Year", y = "Matches", title = "ATP matches per year (full dataset)") +
  theme_bw(base_size = 10)
```



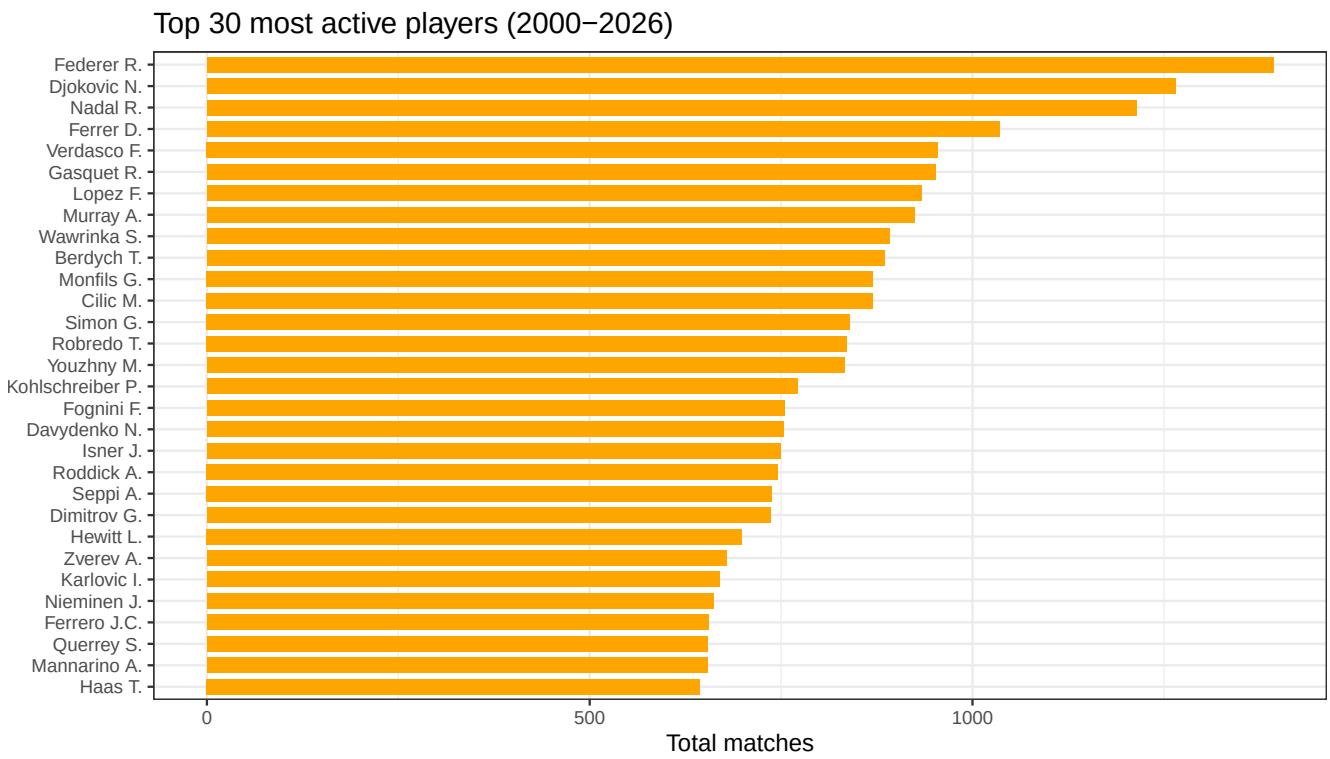

### Surface distribution

```
df_clean %>%
  count(surface) %>%
  mutate(pct = n / sum(n)) %>%
  ggplot(aes(x = reorder(surface, -n), y = pct, fill = surface)) +
  geom_col(width = 0.6, show.legend = FALSE) +
  scale_y_continuous(labels = scales::percent) +
  labs(x = "Surface", y = "Share of matches",
       title = "Match share by surface (full dataset)") +
  theme_bw(base_size = 10)
```




### Player activity (top 30)

```
df_clean %>%
  pivot_longer(c(winner_name, loser_name), values_to = "player") %>%
  count(player, name = "matches") %>%
  slice_max(matches, n = 30) %>%
  ggplot(aes(x = reorder(player, matches), y = matches)) +
  geom_col(fill = "orange", width = 0.7) +
  coord_flip() +
  labs(x = NULL, y = "Total matches", title = "Top 30 most active players (2000-2026)") +
  theme_bw(base_size = 9)
```




# Subsetting for Modelling

## Rationale

Fitting the full dataset (67,000+ matches, 1,000+ players) introduces two problems:

1. **Sparsity.** Most player pairs have never met. A player with only 5 matches has a nearly unidentified $\beta_i$ — the posterior will be almost entirely the prior.
2. **Computational cost.** Stan's HMC scales poorly with the number of parameters. With 1,000 players $\times$ 12 seasons, Model 2 would require ~12,000 parameters.

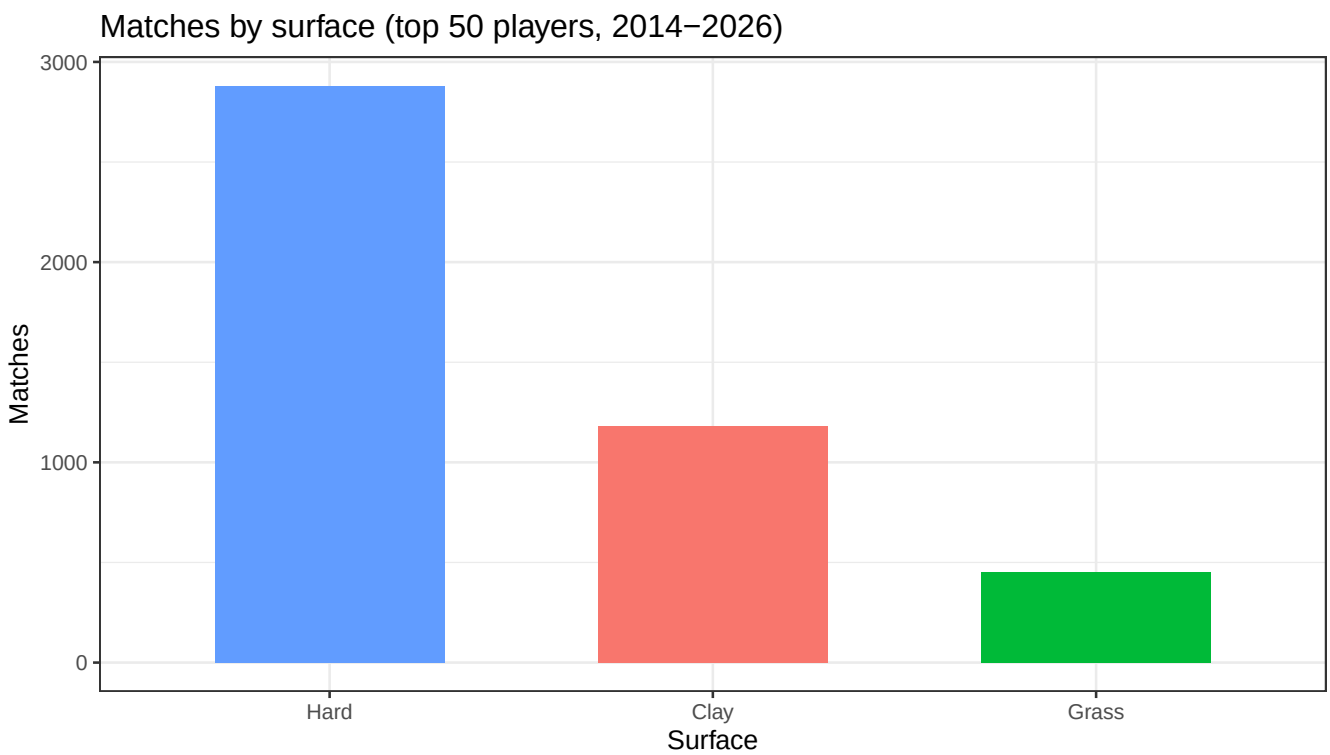
We address both by restricting to **2014–2026** (recent, stable era of the tour) and the **top 50 players** by match count.

```
df_2014 <- df_clean %>% filter(year >= 2014)

player_counts <- df_2014 %>%
  pivot_longer(c(winner_name, loser_name), values_to = "player") %>%
  count(player, name = "total_matches") %>%
  arrange(desc(total_matches))

top_players <- player_counts %>% slice(1:50) %>% pull(player)

df_model <- df_2014 %>%
  filter(winner_name %in% top_players, loser_name %in% top_players)

cat("Modelling subset - matches:", nrow(df_model),
    "| players:", length(top_players), "\n")
```

```
## Modelling subset - matches: 4515 | players: 50
```

### EDA: Modelling subset

```
df_model %>%
  count(surface) %>%
  ggplot(aes(x = reorder(surface, -n), y = n, fill = surface)) +
  geom_col(width = 0.6, show.legend = FALSE) +
  labs(x = "Surface", y = "Matches",
       title = "Matches by surface (top 50 players, 2014-2026)") +
  theme_bw(base_size = 10)
```

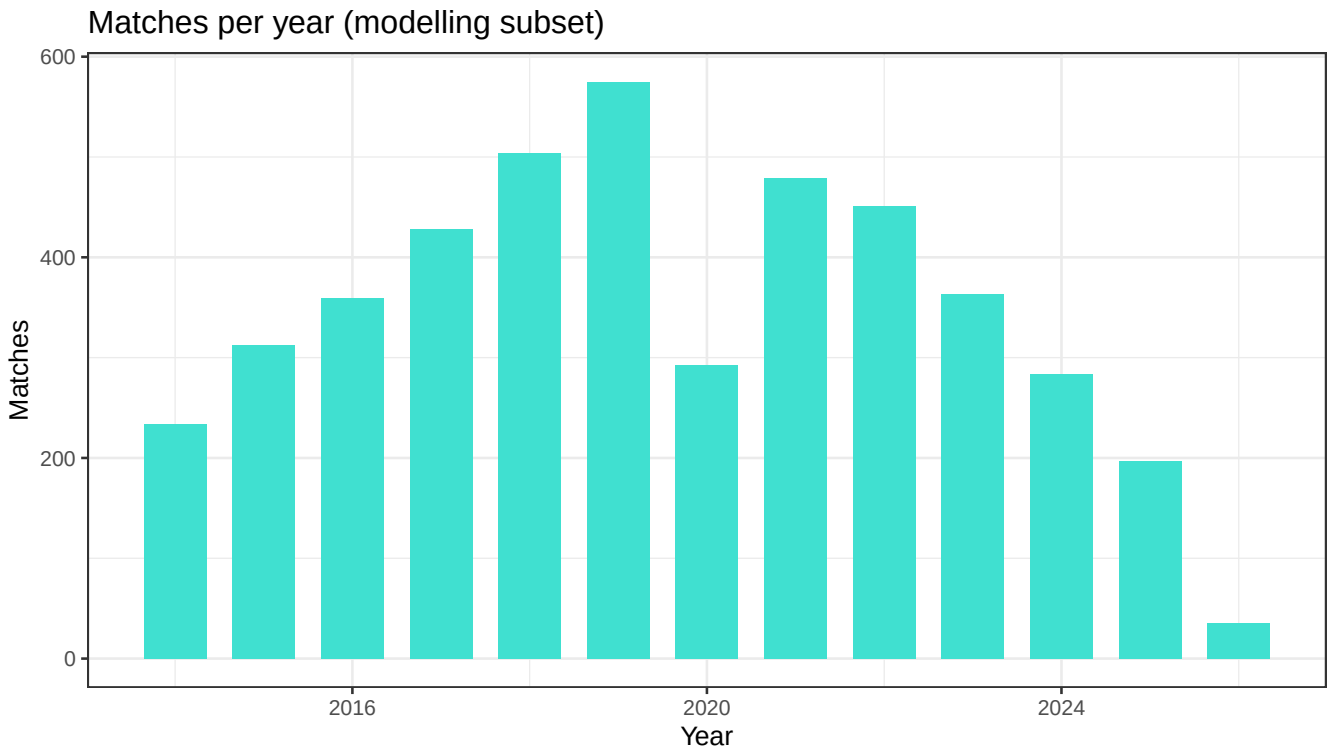



In the original dataset, matches are played on four surface types: hard, clay, grass, and carpet. However, after filtering to matches from 2014 onward and restricting to the most active players, the carpet surface does not appear in the subset.

This is expected, as carpet courts have largely been phased out of professional ATP tournaments. In fact, carpet was removed from the ATP Tour schedule in the late 2000s and replaced by hard courts for consistency and player safety. As a result, there are no recent matches played on carpet within our selected time period, and it does not appear in the filtered dataset.

Therefore, our analysis focuses on the three primary surfaces: hard, clay, and grass.

```
df_model %>%
  count(year) %>%
  ggplot(aes(x = year, y = n)) +
  geom_col(fill = "turquoise", width = 0.7) +
  labs(x = "Year", y = "Matches",
       title = "Matches per year (modelling subset)") +
  theme_bw(base_size = 10)
```

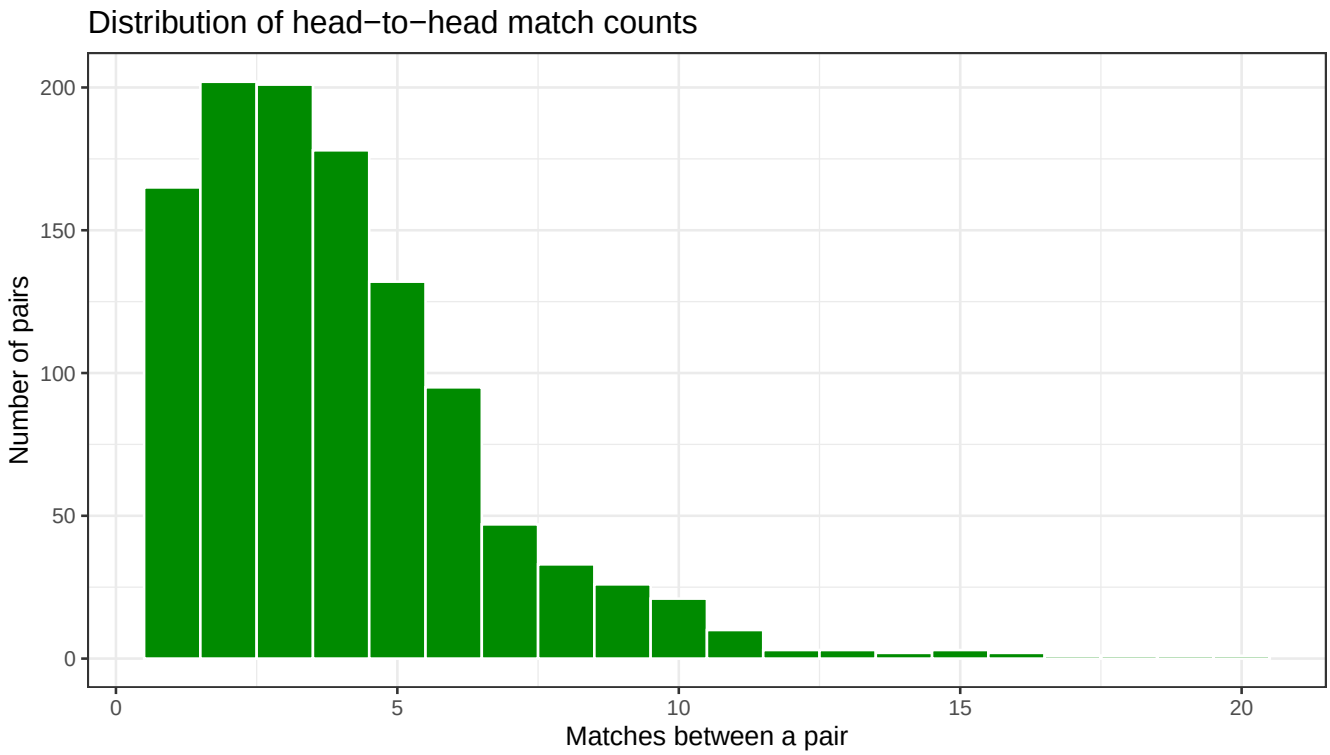



### Match coverage matrix (sparsity check)

```
# How many unique pairs have at least 1 match?
pairs_observed <- df_model %>%
  mutate(pair = paste(pmin(winner_name, loser_name),
                      pmax(winner_name, loser_name), sep = " vs ")) %>%
  distinct(pair) %>%
  nrow()

total_pairs <- choose(length(top_players), 2)

cat(sprintf("Observed pairs: %d / %d possible (%.1f%% coverage)\n",
            pairs_observed, total_pairs,
            100 * pairs_observed / total_pairs))
```

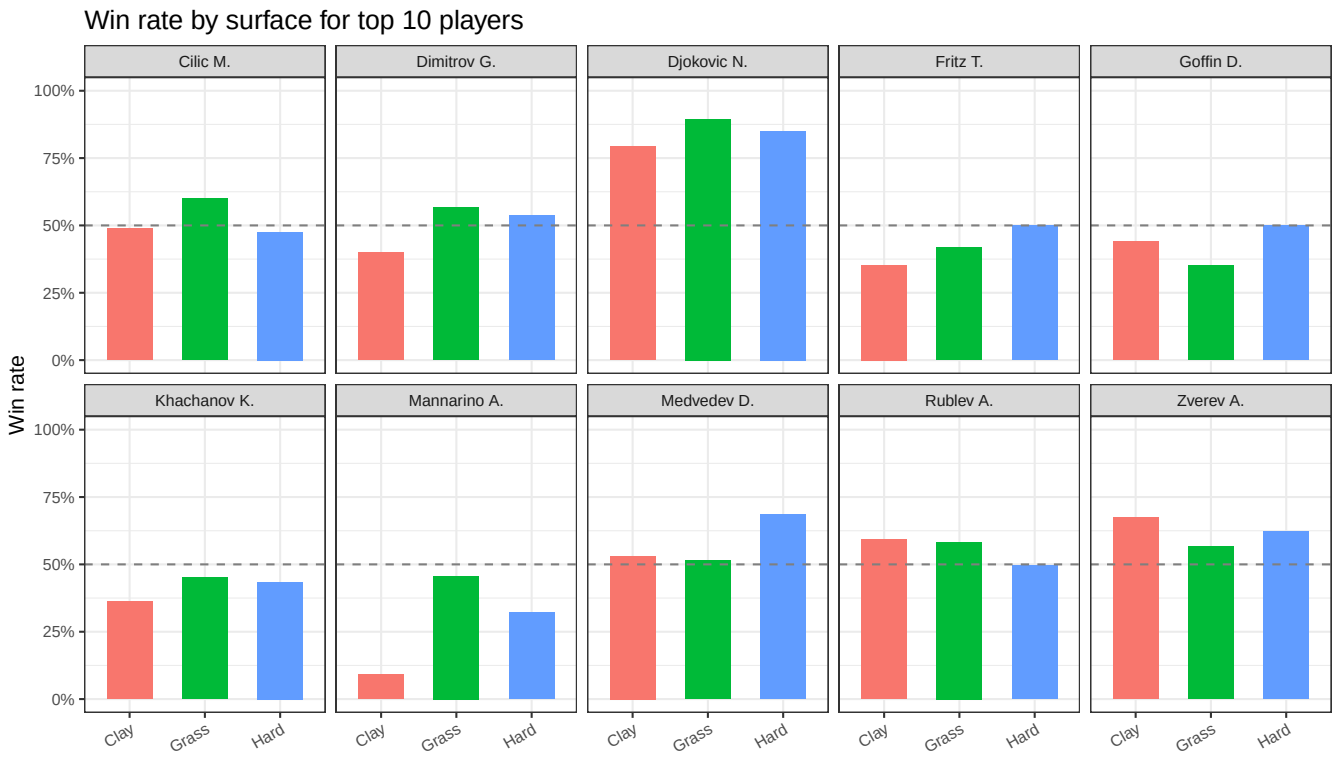
```
## Observed pairs: 1127 / 1225 possible (92.0% coverage)
```

```
df_model %>%
  mutate(pair = paste(pmin(winner_name, loser_name),
                      pmax(winner_name, loser_name), sep = " vs ")) %>%
  count(pair, name = "n_matches") %>%
  ggplot(aes(x = n_matches)) +
  geom_histogram(binwidth = 1, fill = "green4", color = "white") +
  labs(x = "Matches between a pair", y = "Number of pairs",
       title = "Distribution of head-to-head match counts") +
  theme_bw(base_size = 10)
```




### Win rates by surface (top 10 players)

```
top10 <- player_counts %>% slice(1:10) %>% pull(player)

win_rates <- df_model %>%
  filter(winner_name %in% top10 | loser_name %in% top10) %>%
  pivot_longer(c(winner_name, loser_name),
               names_to = "outcome", values_to = "player") %>%
  filter(player %in% top10) %>%
  mutate(won = outcome == "winner_name") %>%
  group_by(player, surface) %>%
  summarise(win_rate = mean(won), n = n(), .groups = "drop") %>%
  filter(n >= 10)

ggplot(win_rates, aes(x = surface, y = win_rate, fill = surface)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_hline(yintercept = 0.5, linetype = "dashed", color = "gray50") +
  facet_wrap(~player, ncol = 5) +
  scale_y_continuous(labels = scales::percent, limits = c(0, 1)) +
  labs(x = NULL, y = "Win rate",
       title = "Win rate by surface for top 10 players") +
  theme_bw(base_size = 8) +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
```




## Save Cleaned Data

Save `df_model` as a CSV so the main report can load it directly without re-running the full wrangling pipeline.

```
write.csv(df_model, "df_model.csv", row.names = FALSE)
cat("Saved df_model.csv with", nrow(df_model), "rows.\n")
```

```
## Saved df_model.csv with 4515 rows.
```